In [19]:
```
# ==========================================
# Cell 1: Environment Setup & Imports
# ==========================================

import pandas as pd
import numpy as np
from datetime import datetime, timedelta

# For reproducibility
np.random.seed(42)

# Display settings for better debugging
pd.set_option('display.max_rows', 50)
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 1000)

print("Environment setup complete.")
```

```
Environment setup complete.
```

In [20]:
```
# ==========================================
# Cell 2: Generate Platform Transactions
# ==========================================

# Configuration
NUM_TRANSACTIONS = 100
START_DATE = datetime(2024, 1, 1)
END_DATE = datetime(2024, 1, 31)

date_range = (END_DATE - START_DATE).days

# Generate base transactions
platform_data = {
    "txn_id": [f"TXN{i:04d}" for i in range(1, NUM_TRANSACTIONS + 1)],
    "txn_date": [START_DATE + timedelta(days=np.random.randint(0, date_range)) for _ in range(NUM_TRANSACTIONS)],
    "amount": np.round(np.random.uniform(100, 10000, NUM_TRANSACTIONS), 2),
    "status": ["SUCCESS"] * NUM_TRANSACTIONS
}
```

```
platform_df = pd.DataFrame(platform_data)

# Sort for realism
platform_df = platform_df.sort_values(by="txn_date").reset_index(drop=True)

print("Platform Transactions Sample:")
display(platform_df.head())

print("\nTotal Transactions:", len(platform_df))
```

```
Platform Transactions Sample:
```

	txn_id	txn_date	amount	status
0	TXN0031	2024-01-01	1044.56	SUCCESS
1	TXN0100	2024-01-02	6838.98	SUCCESS
2	TXN0028	2024-01-02	9158.10	SUCCESS
3	TXN0089	2024-01-02	2544.07	SUCCESS
4	TXN0023	2024-01-02	7142.29	SUCCESS

```
Total Transactions: 100
```

In [21]:

```
# ==========================================
# Cell 3: Generate Bank Settlements + Inject Gaps
# ==========================================

bank_df = platform_df.copy()

# --- Simulate T+1 / T+2 settlement delay ---
bank_df["settlement_date"] = bank_df["txn_date"] + bank_df["txn_date"].apply(
    lambda _: timedelta(days=np.random.choice([1, 2]))
)

# --- 1. Late Settlements (spill into next month) ---
late_indices = np.random.choice(bank_df.index, size=5, replace=False)
bank_df.loc[late_indices, "settlement_date"] = datetime(2024, 2, 2)

# --- 2. Rounding Differences ---
rounding_indices = np.random.choice(bank_df.index, size=5, replace=False)
```

```python
bank_df.loc[rounding_indices, "amount"] += np.random.choice([0.01, -0.01, 0.02], size=5)

# --- 3. Duplicate Entries in Bank ---
duplicate_rows = bank_df.sample(5, random_state=42)
bank_df = pd.concat([bank_df, duplicate_rows], ignore_index=True)

# --- 4. Refund without Original Transaction ---
refund_entries = pd.DataFrame({
    "txn_id": [f"REFUND{i:03d}" for i in range(1, 4)],
    "txn_date": [datetime(2024, 1, 15)] * 3,
    "amount": [-500, -1200, -750],
    "status": ["REFUND"] * 3,
    "settlement_date": [datetime(2024, 1, 16)] * 3
})

bank_df = pd.concat([bank_df, refund_entries], ignore_index=True)

# Shuffle to remove ordering bias
bank_df = bank_df.sample(frac=1, random_state=42).reset_index(drop=True)

print("Bank Settlements Sample:")
display(bank_df.head())

print("\nTotal Bank Records:", len(bank_df))
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[21], line 8
      5 bank_df = platform_df.copy()
      7 # --- Simulate T+1 / T+2 settlement delay ---
----> 8 bank_df["settlement_date"] = bank_df["txn_date"] + bank_df["txn_date"].apply(
      9     lambda _: timedelta(days=np.random.choice([1, 2]))
     10 )
     12 # --- 1. Late Settlements (spill into next month) ---
     13 late_indices = np.random.choice(bank_df.index, size=5, replace=False)

File ~\AppData\Roaming\Python\Python312\site-packages\pandas\core\series.py:4915, in Series.apply(self, func, convert_dtype, ar
gs, by_row, **kwargs)
   4780 def apply(
   4781     self,
   4782     func: AggFuncType,
   (...)
   4787     **kwargs,
   4788 ) -> DataFrame | Series:
   4789     """
   4790     Invoke function on values of Series.
   4791
   (...)
   4906     dtype: float64
   4907     """
   4908     return SeriesApply(
   4909         self,
   4910         func,
   4911         convert_dtype=convert_dtype,
   4912         by_row=by_row,
   4913         args=args,
   4914         kwargs=kwargs,
-> 4915     ).apply()

File ~\AppData\Roaming\Python\Python312\site-packages\pandas\core\apply.py:1427, in SeriesApply.apply(self)
   1424     return self.apply_compat()
   1426 # self.func is Callable
-> 1427 return self.apply_standard()

File ~\AppData\Roaming\Python\Python312\site-packages\pandas\core\apply.py:1507, in SeriesApply.apply_standard(self)
```

```
   1501 # row-wise access
   1502 # apply doesn't have a `na_action` keyword and for backward compat reasons
   1503 # we need to give `na_action="ignore"` for categorical data.
   1504 # TODO: remove the `na_action="ignore"` when that default has been changed in
   1505 #  Categorical (GH51645).
   1506 action = "ignore" if isinstance(obj.dtype, CategoricalDtype) else None
-> 1507 mapped = obj._map_values(
   1508     mapper=curried, na_action=action, convert=self.convert_dtype
   1509 )
   1511 if len(mapped) and isinstance(mapped[0], ABCSeries):
   1512     # GH#43986 Need to do list(mapped) in order to get treated as nested
   1513     #  See also GH#25959 regarding EA support
   1514     return obj._constructor_expanddim(list(mapped), index=obj.index)

File ~\AppData\Roaming\Python\Python312\site-packages\pandas\core\base.py:919, in IndexOpsMixin._map_values(self, mapper, na_ac
tion, convert)
    916 arr = self._values
    918 if isinstance(arr, ExtensionArray):
--> 919     return arr.map(mapper, na_action=na_action)
    921 return algorithms.map_array(arr, mapper, na_action=na_action, convert=convert)

File ~\AppData\Roaming\Python\Python312\site-packages\pandas\core\arrays\_mixins.py:81, in ravel_compat.<locals>.method(self, *
args, **kwargs)
     78 @wraps(meth)
     79 def method(self, *args, **kwargs):
     80     if self.ndim == 1:
---> 81         return meth(self, *args, **kwargs)
     83     flags = self._ndarray.flags
     84     flat = self.ravel("K")

File ~\AppData\Roaming\Python\Python312\site-packages\pandas\core\arrays\datetimelike.py:738, in DatetimeLikeArrayMixin.map(sel
f, mapper, na_action)
    734 @ravel_compat
    735 def map(self, mapper, na_action=None):
    736     from pandas import Index
--> 738     result = map_array(self, mapper, na_action=na_action)
    739     result = Index(result)
    741     if isinstance(result, ABCMultiIndex):

File ~\AppData\Roaming\Python\Python312\site-packages\pandas\core\algorithms.py:1743, in map_array(arr, mapper, na_action, conv
ert)
```

```
   1741 values = arr.astype(object, copy=False)
   1742 if na_action is None:
-> 1743     return lib.map_infer(values, mapper, convert=convert)
   1744 else:
   1745     return lib.map_infer_mask(
   1746         values, mapper, mask=isna(values).view(np.uint8), convert=convert
   1747     )

File lib.pyx:2972, in pandas._libs.lib.map_infer()

Cell In[21], line 9, in <lambda>(_)
      5 bank_df = platform_df.copy()
      7 # --- Simulate T+1 / T+2 settlement delay ---
      8 bank_df["settlement_date"] = bank_df["txn_date"] + bank_df["txn_date"].apply(
----> 9     lambda _: timedelta(days=np.random.choice([1, 2]))
     10 )
     12 # --- 1. Late Settlements (spill into next month) ---
     13 late_indices = np.random.choice(bank_df.index, size=5, replace=False)

TypeError: unsupported type for timedelta days component: numpy.int32
```

In [22]:

```python
# ============================================
# Cell 3: Generate Bank Settlements + Inject Gaps (FIXED)
# ============================================

bank_df = platform_df.copy()

# --- Simulate T+1 / T+2 settlement delay (VECTORISED) ---
settlement_delay = np.random.choice([1, 2], size=len(bank_df))
bank_df["settlement_date"] = bank_df["txn_date"] + pd.to_timedelta(settlement_delay, unit="D")

# --- 1. Late Settlements (spill into next month) ---
late_indices = np.random.choice(bank_df.index, size=5, replace=False)
bank_df.loc[late_indices, "settlement_date"] = datetime(2024, 2, 2)

# --- 2. Rounding Differences ---
rounding_indices = np.random.choice(bank_df.index, size=5, replace=False)
bank_df.loc[rounding_indices, "amount"] += np.random.choice([0.01, -0.01, 0.02], size=5)

# --- 3. Duplicate Entries in Bank ---
duplicate_rows = bank_df.sample(5, random_state=42)
```

```
bank_df = pd.concat([bank_df, duplicate_rows], ignore_index=True)

# --- 4. Refund without Original Transaction ---
refund_entries = pd.DataFrame({
    "txn_id": [f"REFUND{i:03d}" for i in range(1, 4)],
    "txn_date": [datetime(2024, 1, 15)] * 3,
    "amount": [-500, -1200, -750],
    "status": ["REFUND"] * 3,
    "settlement_date": [datetime(2024, 1, 16)] * 3
})

bank_df = pd.concat([bank_df, refund_entries], ignore_index=True)

# Shuffle to remove ordering bias
bank_df = bank_df.sample(frac=1, random_state=42).reset_index(drop=True)

print("Bank Settlements Sample:")
display(bank_df.head())

print("\nTotal Bank Records:", len(bank_df))
```

```
Bank Settlements Sample:
```

	txn_id	txn_date	amount	status	settlement_date
0	TXN0070	2024-01-26	3627.35	SUCCESS	2024-01-27
1	TXN0088	2024-01-04	2613.77	SUCCESS	2024-01-05
2	TXN0023	2024-01-02	7142.29	SUCCESS	2024-01-04
3	TXN0040	2024-01-27	8504.25	SUCCESS	2024-01-28
4	TXN0071	2024-01-21	9077.60	SUCCESS	2024-01-22

```
Total Bank Records: 108
```

In [25]:

```
# ==========================================
# Cell 4: Reconciliation Engine
# ==========================================

# --- Merge platform and bank data ---
recon_df = pd.merge(
```

```
    platform_df,
    bank_df,
    on="txn_id",
    how="outer",
    suffixes=("_platform", "_bank"),
    indicator=True
)

# --- 1. Missing Settlements (present in platform, not in bank) ---
missing_settlements = recon_df[recon_df["_merge"] == "left_only"]

# --- 2. Unexpected Bank Entries (present in bank, not in platform) ---
unexpected_bank = recon_df[recon_df["_merge"] == "right_only"]

# --- 3. Amount Mismatches (only where both exist) ---
amount_mismatch = recon_df[
    (recon_df["_merge"] == "both") &
    (np.round(recon_df["amount_platform"], 2) != np.round(recon_df["amount_bank"], 2))
]

# --- 4. Duplicate Entries in Bank ---
duplicate_bank = bank_df[bank_df.duplicated(subset=["txn_id"], keep=False)]

# --- 5. Late Settlements (settled after Jan 31) ---
month_end = datetime(2024, 1, 31)

late_settlements = recon_df[
    (recon_df["_merge"] == "both") &
    (recon_df["settlement_date"] > month_end)
]

# --- Summary Metrics ---
summary = {
    "missing_settlement": len(missing_settlements),
    "unexpected_bank_entries": len(unexpected_bank),
    "amount_mismatches": len(amount_mismatch),
    "duplicate_entries": len(duplicate_bank),
    "late_settlements": len(late_settlements)
}
```

```
print("Reconciliation Summary:")
print(summary)
```

```
Reconciliation Summary:
{'missing_settlement': 0, 'unexpected_bank_entries': 3, 'amount_mismatches': 5, 'duplicate_entries': 10, 'late_settlements': 8}
```

In [26]:

```
# ==========================================
# Cell 5: Corrected Reconciliation (Transaction-Level)
# ==========================================

# --- Step 1: Deduplicate Bank Data (keep first occurrence) ---
bank_dedup = bank_df.drop_duplicates(subset=["txn_id"], keep="first").copy()

# --- Step 2: Merge again using clean bank data ---
recon_clean = pd.merge(
    platform_df,
    bank_dedup,
    on="txn_id",
    how="outer",
    suffixes=("_platform", "_bank"),
    indicator=True
)

# --- Step 3: Recompute Metrics ---

# Missing settlements
missing_settlements_clean = recon_clean[recon_clean["_merge"] == "left_only"]

# Unexpected bank entries
unexpected_bank_clean = recon_clean[recon_clean["_merge"] == "right_only"]

# Amount mismatches
amount_mismatch_clean = recon_clean[
    (recon_clean["_merge"] == "both") &
    (np.round(recon_clean["amount_platform"], 2) != np.round(recon_clean["amount_bank"], 2))
]

# Late settlements (TRUE count now)
month_end = datetime(2024, 1, 31)

late_settlements_clean = recon_clean[
```

```
    (recon_clean["_merge"] == "both") &
    (recon_clean["settlement_date"] > month_end)
]

# Duplicate entries (still from original bank_df)
duplicate_bank_clean = bank_df[bank_df.duplicated(subset=["txn_id"], keep=False)]

# --- Final Clean Summary ---
clean_summary = {
    "missing_settlement": len(missing_settlements_clean),
    "unexpected_bank_entries": len(unexpected_bank_clean),
    "amount_mismatches": len(amount_mismatch_clean),
    "duplicate_entries": len(duplicate_bank_clean),
    "late_settlements": len(late_settlements_clean)
}

print("Clean Reconciliation Summary (Corrected):")
print(clean_summary)
```

```
Clean Reconciliation Summary (Corrected):
{'missing_settlement': 0, 'unexpected_bank_entries': 3, 'amount_mismatches': 5, 'duplicate_entries': 10, 'late_settlements': 8}
```

In [27]:

```
# ==========================================
# Cell 6: Correct Late Settlement Logic (Txn-Level Truth)
# ==========================================

# Get latest settlement per txn_id
latest_settlement = bank_df.groupby("txn_id")["settlement_date"].max().reset_index()

# Merge with platform to keep only valid txns
late_check = pd.merge(
    platform_df[["txn_id"]],
    latest_settlement,
    on="txn_id",
    how="left"
)

# Late settlements = any txn settled after month end
month_end = datetime(2024, 1, 31)

true_late_settlements = late_check[
```

```
    late_check["settlement_date"] > month_end
]

print("True Late Settlements Count:", len(true_late_settlements))
```

```
True Late Settlements Count: 8
```

In [28]:

```
# Debug: Check actual unique late txn_ids

late_txns = bank_df[bank_df["settlement_date"] > datetime(2024, 1, 31)]

unique_late_txns = late_txns["txn_id"].nunique()

print("Unique Late txn_ids:", unique_late_txns)
print("\nLate txn_ids:")
print(late_txns["txn_id"].unique())
```

```
Unique Late txn_ids: 8

Late txn_ids:
['TXN0026' 'TXN0017' 'TXN0050' 'TXN0032' 'TXN0047' 'TXN0081' 'TXN0046'
 'TXN0097']
```

In [30]:

```
# ============================================
# Cell 8: Detailed Outputs (For Submission Evidence)
# ============================================

print("Missing Settlements:")
display(missing_settlements_clean)

print("\nUnexpected Bank Entries:")
display(unexpected_bank_clean)

print("\nAmount Mismatches:")
display(amount_mismatch_clean)

print("\nDuplicate Bank Entries:")
display(duplicate_bank_clean.head(10))

print("\nLate Settlements (True txn-level):")
display(true_late_settlements)
```

```
Missing Settlements:
```

txn_id	txn_date_platform	amount_platform	status_platform	txn_date_bank	amount_bank	status_bank	settlement_date	_merge

```
Unexpected Bank Entries:
```

	txn_id	txn_date_platform	amount_platform	status_platform	txn_date_bank	amount_bank	status_bank	settlement_date	_merge
0	REFUND001	NaT	NaN	NaN	2024-01-15	-500.0	REFUND	2024-01-16	right_only
1	REFUND002	NaT	NaN	NaN	2024-01-15	-1200.0	REFUND	2024-01-16	right_only
2	REFUND003	NaT	NaN	NaN	2024-01-15	-750.0	REFUND	2024-01-16	right_only

```
Amount Mismatches:
```

	txn_id	txn_date_platform	amount_platform	status_platform	txn_date_bank	amount_bank	status_bank	settlement_date	_merge
21	TXN0019	2024-01-24	4009.33	SUCCESS	2024-01-24	4009.32	SUCCESS	2024-01-25	both
50	TXN0048	2024-01-15	2097.02	SUCCESS	2024-01-15	2097.03	SUCCESS	2024-01-17	both
75	TXN0073	2024-01-20	6512.13	SUCCESS	2024-01-20	6512.15	SUCCESS	2024-01-21	both
90	TXN0088	2024-01-04	2613.76	SUCCESS	2024-01-04	2613.77	SUCCESS	2024-01-05	both
92	TXN0090	2024-01-30	6993.41	SUCCESS	2024-01-30	6993.42	SUCCESS	2024-01-31	both

```
Duplicate Bank Entries:
```

	txn_id	txn_date	amount	status	settlement_date
3	TXN0040	2024-01-27	8504.25	SUCCESS	2024-01-28
7	TXN0075	2024-01-15	3590.43	SUCCESS	2024-01-16
27	TXN0040	2024-01-27	8504.25	SUCCESS	2024-01-28
29	TXN0045	2024-01-15	7135.56	SUCCESS	2024-01-17
35	TXN0075	2024-01-15	3590.43	SUCCESS	2024-01-16
39	TXN0057	2024-01-19	9434.25	SUCCESS	2024-01-20
66	TXN0012	2024-01-23	9656.03	SUCCESS	2024-01-25
75	TXN0045	2024-01-15	7135.56	SUCCESS	2024-01-17
84	TXN0057	2024-01-19	9434.25	SUCCESS	2024-01-20
107	TXN0012	2024-01-23	9656.03	SUCCESS	2024-01-25

```
Late Settlements (True txn-level):
```

	txn_id	settlement_date
12	TXN0017	2024-02-02
35	TXN0032	2024-02-02
41	TXN0081	2024-02-02
51	TXN0097	2024-02-02
55	TXN0050	2024-02-02
95	TXN0026	2024-02-01
98	TXN0046	2024-02-01
99	TXN0047	2024-02-01

In [31]:
```
# ==========================================
# Cell 9: Final Summary Report
```

```
# ==========================================

final_report = {
    "Missing Settlements": len(missing_settlements_clean),
    "Unexpected Bank Entries": len(unexpected_bank_clean),
    "Amount Mismatches": len(amount_mismatch_clean),
    "Duplicate Entries (Raw Bank)": len(duplicate_bank_clean),
    "Late Settlements (Txn-level)": len(true_late_settlements)
}

report_df = pd.DataFrame(final_report.items(), columns=["Metric", "Count"])

print("Final Reconciliation Report:")
display(report_df)
```

```
Final Reconciliation Report:
```

	Metric	Count
0	Missing Settlements	0
1	Unexpected Bank Entries	3
2	Amount Mismatches	5
3	Duplicate Entries (Raw Bank)	10
4	Late Settlements (Txn-level)	8

In [32]:

```
# ==========================================
# Cell 10: Validation Checks
# ==========================================

assert len(unexpected_bank_clean) == 3, "Unexpected bank entries mismatch"
assert len(amount_mismatch_clean) == 5, "Amount mismatch count incorrect"
assert len(duplicate_bank_clean) == 10, "Duplicate detection incorrect"

print("All validation checks passed.")
```

```
All validation checks passed.
```

In [ ]:

```

```